# NOSE-CONE DESIGN TOOL: USER GUIDE

**Fabio Feig - 315169**




Date: October 27, 2026

Doc ID: 2027_C_FD_REPORT-3_NOSE-CONE-TOOL-USER-GUIDE

**Python package** `nosecone` **– version 1.0**

EPFL ROCKET TEAM

# Contents

**Quick start** 1

**1 What the Tool Does, and What It Does Not** 1

**2 Installation and Requirements** 2
2.1 Requirements . . . 2
2.2 Installation . . . 3
2.3 Checking the installation . . . 3

**3 Repository Structure** 3

**4 The Configuration File** 4
4.1 How the file works . . . 4
4.2 The parameters you change . . . 5
4.3 Structured parameters . . . 5

**5 Running Studies** 6
5.1 Commands . . . 6
5.2 Worked example 1: a single design . . . 6
5.3 Worked example 2: a parameter sweep . . . 7
5.4 Other cases . . . 7

**6 Reading the Outputs** 8
6.1 The output folder . . . 8
6.2 Tables . . . 8
6.3 Plots . . . 8

**7 Extending the Tool** 11
7.1 Adding a profile family . . . 11
7.2 Adding a plot . . . 11
7.3 Adding a parameter or changing a model . . . 11
7.4 Checking a change . . . 12

**A Verification Record** 12

**B Troubleshooting** 12

# Quick start

**From zero to the Weisshorn results in four commands** (Windows; see Section 2 for macOS/Linux):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m nosecone run configs/weisshorn.yaml
```

Then open `output/weisshorn/summary.md`: it lists every nose of the study, best first, with its wave drag, apogee, mass, payload room and static-margin change. Figures are in `output/weisshorn/figures/`, CAD files (CSV, STL, STEP) in `output/weisshorn/cad/`.

**To study another rocket**, copy `configs/example_new_rocket.yaml`, change the values of your vehicle (diameter, length, masses, engine, fins) and the noses you want to compare, and run it with the same command. You never need to edit the Python code for that: every input is in the YAML file. The parameters are described in Section 4, the outputs in Section 6.

**How this guide is organised.**

- Section 1: what the tool computes, with which models, and where these models stop being valid.
- Section 2: installation and requirements.
- Section 3: the repository and what each module does.
- Section 4: the configuration file and the parameters you change.
- Section 5: running a single design and a parameter sweep, with worked examples for our 180 mm rocket.
- Section 6: how to read the plots and tables.
- Section 7: adding a profile family, a plot, a parameter, or changing a model.

# 1 What the Tool Does, and What It Does Not

The `nosecone` package compares nose cones for a supersonic sounding rocket (*profile*, *fineness* $L/D$ and *tip radius*) and exports the CAD geometry of the selected one. It reproduces 2027_C_FD_REPORT-1_NOSE-CONE exactly, but every vehicle-specific number is an input, so it serves any ERT rocket.

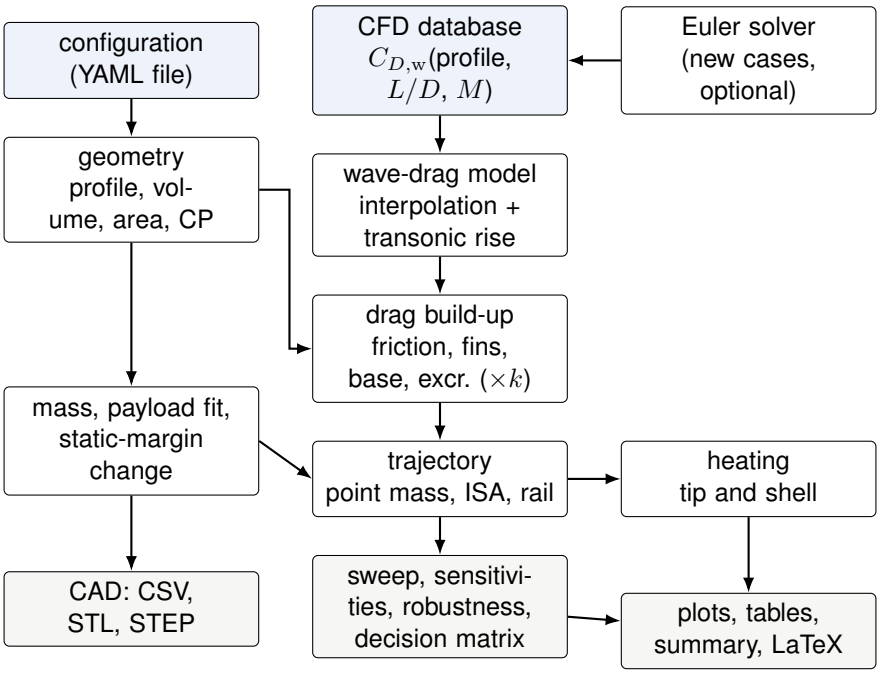



Figure 1: Computation chain of one run (`python -m nosecone run`). Shaded blue: inputs; grey: outputs.

**Computed for every nose** (profile $\times$ fineness $\times$ tip radius): geometry (volume, wetted area, shoulder angle, Barrowman CP, payload room), wave drag $C_{D,\mathrm{w}}(M)$ from the CFD database, apogee of the full vehicle, nose mass and static-margin change against a baseline nose. **For the study**: apogee sensitivities, robustness of the ranking, cost of a blunt tip, tip and shell temperatures, a decision matrix and the CAD files.

**Limits.**

- **Wave drag**: inviscid Euler CFD at zero incidence, tabulated at $M = 1.1$–1.6 and $L/D = 2.5$–5 for the sweep profiles; held constant outside. Below $M = 1.1$ an assumed transonic ramp (the weakest part; replace with RANS data).
- **Rest of the vehicle**: empirical drag build-up scaled by one factor $k$, calibrated on a known apogee.
- **Trajectory**: 2D point mass, ISA below 11 km, no wind. **Stability**: Barrowman, use for *changes* of static margin only.
- **Not done**: structure, viscous or transonic CFD, angle of attack, 6-DOF flight, shape optimisation. A profile without CFD data is skipped unless `aerodynamics.compute_missing` is true.

`tests/verify_reference.py` checks that the Weisshorn results are still reproduced (Appendix A); run it after any code change.

# 2 Installation and Requirements

## 2.1 Requirements

- **Python 3.11 or newer** (developed with 3.14) and `git` to get the repository.

- The Python packages of `requirements.txt`: NumPy, SciPy, pandas, Matplotlib, PyYAML and, optionally, gmsh. The first four are pinned to the versions that produced the Weisshorn report, so that the results are reproduced bit for bit; newer versions give the same results to about $10^{-12}$.
- **Optional: a LaTeX installation** (MiKTeX on Windows, TeX Live on Linux/macOS). With it, the plots use LaTeX fonts and are identical to the report figures; without it, Matplotlib's built-in renderer is used automatically and the plots look slightly different but contain the same information.
- **Optional: gmsh** (installed by `requirements.txt`, ≈70 MB) for the STEP export. Without it the STEP file is skipped with a message; the CSV and STL files are always written.

## 2.2 Installation

Get the repository (private: ask for access, or download it as a zip from GitHub) and install the packages in a virtual environment, so that they do not interfere with other Python projects:

```
git clone https://github.com/Landonowins/weisshorn-nose-cone.git
cd weisshorn-nose-cone
python -m venv .venv
.venv\Scripts\activate              # Windows (PowerShell or cmd)
source .venv/bin/activate           # macOS / Linux
pip install -r requirements.txt
```

The virtual environment must be activated (the prompt shows `(.venv)`) in every new terminal before using the tool. All commands of this guide are run from the repository folder.

## 2.3 Checking the installation

```
python -m nosecone --version                    # prints: nosecone 1.0.0
python -m nosecone validate configs/weisshorn.yaml
python tests/verify_reference.py                # about 2 minutes; must end with "All
    outputs match"
```

**Common problems.** *"No module named nosecone"*: you are not in the repository folder, or the virtual environment is not active. *LaTeX errors in the plots*: set `output.text_renderer: mathtext` or install the missing LaTeX packages (MiKTeX installs them on the fly if allowed). *"STEP skipped"*: `pip install gmsh`.

# 3 Repository Structure

```
weisshorn-nose-cone/
|-- nosecone/              the Python package (all the code)
|-- configs/               weisshorn.yaml + worked examples
|-- data/                  CFD database (euler_results.csv) and wall-pressure files
|-- plugins/               example plugin (Section 7.1)
|-- docs/user_guide/       this guide
|-- report/, report_650/   the nose-cone design reports
|-- CFD/                   Fluent workflow and runs (REPORT-4)
|-- tests/                 verify_reference.py + reference outputs
|-- output/                created by the tool (not in git)
```

Table 1: Modules of the `nosecone` package, in the order of the computation chain. In Python, `help(nosecone.<module>.<function>)` prints the documentation of any function.

| Module | Job |
|---|---|
| `config` | Read, complete and check the YAML file; every parameter with unit, range and default |
| `constants` | Physical constants and model coefficients (the only fixed numbers) |
| `geometry` | Profile families, blunting, volume, area, CP, payload length |
| `euler`, `cfd` | Axisymmetric Euler solver; CFD studies and the wave-drag database |
| `aerodynamics` | Wave-drag model: interpolation in $M$ and $L/D$, transonic rise |
| `flight` | Atmosphere, drag build-up, trajectory, calibration of $k$ |
| `stability`, `heating` | Barrowman stability and nose mass; tip and shell temperatures |
| `design` | The study: sweep, sensitivities, robustness, bluntness, decision matrix |
| `export` | Profile CSV, STL and STEP files |
| `plotting`, `reporting` | All figures; summary and LaTeX tables |
| `pipeline`, `cli` | Runs everything in order; command line |

# 4 The Configuration File

## 4.1 How the file works

A study is one YAML file (`configs/weisshorn.yaml` is the complete, commented example): *sections* contain `name: value` lines indented by two spaces, lists are `[a, b, c]`, `#` starts a comment.

- **Only what differs from the default is needed**; **units are SI** (lengths in metres: `diameter: 0.180`, not 180); relative paths are relative to the file.
- **Inheritance**: `base: weisshorn.yaml` takes every value of that file and overrides only what is listed; a list replaces the inherited list.
- **Checking**: `python -m nosecone validate my.yaml` stops at the first problem with a precise message (e.g. `unknown parameter 'nose.profils' (did you mean 'profiles'?)`); `--show` prints every value used. The run saves them as `config_used.yaml`.
- **All parameters**: `python -m nosecone describe-config` lists every parameter with unit, valid range and default (`--basic` hides the advanced numerical settings).

## 4.2 The parameters you change

Table 2 lists what a new rocket or a new study needs (`configs/example_new_rocket.yaml` sets exactly these). Everything else has a sensible default.

Table 2: Main parameters. Default values are those of Weisshorn.

| Parameter | Unit | Meaning |
|---|---|---|
| `project.name`, `project.output_dir` | | study name (plots, file names); output folder |
| `vehicle.diameter` | m | body diameter $D$ (nose base) |
| `vehicle.length`, `reference_nose_length` | m | total length; the nose length included in it |
| `vehicle.mass_wet`, `mass_dry` | kg | lift-off and burn-out mass |
| `vehicle.nozzle_exit_diameter` | m | removed from the base area while burning |
| `engine.thrust`, `burn_time` | N, s | constant thrust and burn duration |
| `engine.total_impulse`, `specific_impulse` | N s, s | set the propellant mass |
| `fins.count`, `root_chord`, `tip_chord`, `span`, `sweep`, `thickness` | –, m | trapezoidal fins |
| `launch.rail_length`, `rail_elevation`, `site_altitude` | m, °, m | launch rail and site |
| `flight.calibrate`, `target_apogee` | –, m | scale the non-nose drag by $k$ so that the baseline nose reaches this apogee |
| `flight.drag_factor` | – | $k$ when `calibrate` is false |
| `nose.profiles`, `fineness`, `tip_radius_ratios` | – | the sweep: profiles $\times$ $L/D$ $\times$ $r_n/R$ |
| `nose.baseline_profile`, `baseline_fineness` | – | reference nose for apogee and static-margin changes |
| `nose.design_profile`, `design_fineness`, `design_tip_radius_ratio` | – | selected design (CAD, heating, final drawing) |
| `nose.shoulder_length` | m | cylindrical shoulder behind the curved part |
| `payload.enabled`, `length`, `required_radius` | –, m, m | payload fit in the nose |
| `structure.shell_areal_density`, `tip_mass` | kg/m$^2$, kg | nose mass model |
| `output.plots`, `output.latex` | – | `[all]` or a list of plots (Section 6.3); write LaTeX tables |

## 4.3 Structured parameters

**Profiles.** Built-in: `cone`, `ogive`, `parabolic`, `power050`, `power075`, `lvhaack`, `vonkarman`. Other members of a family (`python -m nosecone profiles` lists them) go in `nose.custom_profiles`:

```
nose:
  custom_profiles:
    power060: {family: power, n: 0.6, label: "Power $n=0.6$"}
  profiles: [vonkarman, power060]
```

A custom profile has no CFD data until it is computed (`aerodynamics.compute_missing: true`).

**Robustness scenarios.** A name plus changes to the nominal model (`transonic_onset`, `transonic_factor`, `drag_factor_scale`, `mass_model`, `ground_temperature`):

```
robustness:
  scenarios:
    - {name: nominal}
    - {name: "transonic peak +20 %", transonic_factor: 1.2}
```

**Decision matrix.** `trade_matrix.weights` (e.g. `{performance: 4, integration: 2, manufacture: 2, stability: 1}`) are normalised to sum to one; `manufacture` scores are your judgement, the others are computed.

**CFD studies.** `cfd.studies`: a list of `{name, profiles, fineness, machs}` (optional `tip_radius_ratios`, `grid`, `max_iter`); see Section 5.4.

# 5 Running Studies

## 5.1 Commands

Everything goes through `python -m nosecone <command>` (`--help` lists the options of each).

Table 3: Commands. Typical run times on a laptop with the Weisshorn database.

| Command | What it does | Time |
|---|---|---|
| `run CONFIG` | The whole study: sweep, sensitivities, robustness, heating, CAD, plots, tables. Options: `--plots trade,cdw`, `--no-plots`, `--latex`, `--output DIR` | 1–5 min |
| `export CONFIG` | Only the CAD files of the selected design | seconds |
| `validate CONFIG` | Check the file; `--show` prints all values used | instant |
| `describe-config` | Every parameter with unit, range and default | instant |
| `profiles` | Profile families and built-in profiles | instant |
| `cfd CONFIG` | Run the CFD studies of `cfd.studies` (`--dry-run` lists the jobs) | 3–10 min per geometry |
| `rebuild-database CONFIG` | Rebuild the database CSV from the wall-pressure files | 1 min |

A run prints its progress (also in `run.log`); when it finishes, start with `summary.md`.

## 5.2 Worked example 1: a single design

*How does a Von Kármán nose of 810 mm ($L/D = 4.5$) with a 4.5 mm tip perform on Weisshorn?* `configs/example_single_design.yaml` inherits the vehicle and restricts the sweep to that nose (main lines):

```
base: weisshorn.yaml
project: {name: Weisshorn single design, output_dir: ../output/example_single_design}
nose:
  profiles: [vonkarman]
```

```
  fineness: [4.5]
  design_profile: vonkarman
  design_fineness: 4.5
  design_tip_radius_ratio: 0.05  # 4.5 mm spherical tip (r_n / R)
robustness: {enabled: false}
trade_matrix: {enabled: false}
```

`summary.md` gives the answer (some columns left out):

```
| shape     | fineness | cd_w_14 | apogee | d_apogee | mass_g | l3U_mm | bay_mm | dSM_cal |
| vonkarman | 4.5      | 0.0425  | 9060   | +60      | 354    | 254    | 907    | -0.036  |
```

With $k = 0.56$ calibrated so that the baseline nose (Von Kármán, $L/D = 4$) reaches 9 km, this nose flies 60 m higher, weighs 354 g, houses 254 mm of the payload in its curved part and lowers the static margin by 0.036 calibre. The CAD files are in `cad/`.

## 5.3 Worked example 2: a parameter sweep

*Which of power-$\frac{1}{2}$, LV-Haack and Von Kármán is best, and how long should the nose be?* Every list in `nose.profiles`, `fineness` or `tip_radius_ratios` is swept (here $3 \times 4$ noses):

```
base: weisshorn.yaml
project: {name: Weisshorn sweep, output_dir: ../output/example_sweep}
nose:
  profiles: [power050, lvhaack, vonkarman]
  fineness: [3.5, 4.0, 4.5, 5.0]
output:
  plots: [cdw, trade, robustness, gallery, schematics]
```

```
| shape     | fineness | cd_w_14 | apogee | d_apogee | mass_g | l3U_mm | bay_mm | dSM_cal |
| vonkarman | 5        | 0.0350  | 9099   | +99      | 386    | 282    | 968    | -0.071  |
| power050  | 5        | 0.0387  | 9075   | +75      | 394    | 241    | 1009   | -0.067  |
| vonkarman | 4.5      | 0.0425  | 9060   | +60      | 354    | 254    | 907    | -0.036  |
| ...
```

From $L/D = 4$ up, Von Kármán flies highest; LV-Haack is 63–81 m lower but houses more payload. Each extra 90 mm of length gains less (+85, +60, +39 m), and the `robustness` plot shows that the ranking holds in every modelling scenario.

## 5.4 Other cases

- **The full Weisshorn study**: `configs/weisshorn.yaml` (seven profiles $\times$ five fineness ratios, all plots, the report's LaTeX tables).
- **Another rocket**: copy `configs/example_new_rocket.yaml`, fill in the parameters of Table 2, then `validate` and `run`. Without a trusted reference apogee, set `flight.calibrate: false` ($k = 1$): apogees are less accurate, comparisons between noses still hold. The CFD database is non-dimensional, so it holds for any diameter.
- **New CFD cases**: either `aerodynamics.compute_missing: true` (missing profiles are solved before the sweep), or list them in `cfd.studies`, run `cfd CONFIG` then `rebuild-database CONFIG`. One geometry with four Mach numbers takes 5–10 minutes per CPU core.

# 6 Reading the Outputs

## 6.1 The output folder

```
output/<study>/
|-- summary.md          the whole sweep in one table, robustness, heating, design
|-- run.log             what the run did
|-- config_used.yaml    every value used, defaults included
|-- tables/             CSV/JSON/TXT results
|-- figures/            one PDF + PNG per plot
|-- cad/                profile CSV, STL and STEP files (mm, x from the tip)
|-- latex/              LaTeX tables and macros (if output.latex)
```

## 6.2 Tables

`trade_table.csv` (and the table of `summary.md`) has one row per nose:

| Column | Meaning |
|---|---|
| `shape`, `fineness` | profile and $L/D$ |
| `L_mm`, `V_L`, `S_m2`, `mass_g` | length [mm], volume [L], wetted area [m$^2$], shell + tip mass [g] |
| `cd_w_11` … | wave drag at $M = 1.1$, 1.2, 1.4, 1.6 ($A_{\mathrm{ref}} = \pi D^2/4$) |
| `apogee`, `d_apogee` | apogee [m] and its change against the baseline nose |
| `l3U_mm`, `bay_mm` | curved length wide enough for the payload; nose-bay length [mm] |
| `dSM_cal` | static-margin change against the baseline [calibres]; negative = less stable |

Other tables in `tables/`: `robustness.csv`, `catalogue.csv` (every profile at the design fineness), `trade_matrix.csv`, `bluntness.csv`, `heating.json`, `design.csv`, `calibration.txt` ($k$) and `sensitivity.txt`. `cad/` holds the profile CSV, the STL of every profile and the STL and STEP of the design (the STEP is the file for SpaceClaim/Fluent). `latex/` holds the report's tables and macros, to `\input{}` so that the text always matches the results.

## 6.3 Plots

Each profile keeps the same colour, line style and marker in every plot.

| Plot | What it shows |
|---|---|
| `profiles` | profiles to scale at the baseline fineness, with the payload radius; tip zoom |
| `schematics` | how each profile is constructed (shoulder, generator, tip) |
| `cdw` | wave drag against Mach (CFD points, model line) and against fineness (Figure 3) |
| `cp` | wall pressure coefficient along the nose: why a profile has low drag |
| `validation` | CFD solver checks: Taylor–Maccoll cones, grid convergence |
| `trajectory` | Mach against time; share of the momentum lost to drag per Mach interval (Figure 4) |
| `breakdown` | drag of the whole vehicle split by component |
| `trade` | apogee change, payload length, static-margin change against fineness (Figure 2) |
| `robustness` | apogee of each profile against the reference in every scenario |
| `bluntness` | wave-drag increase and tip heating against tip radius |
| `thermal` | shell and tip temperatures, nominal and hot day |
| `final` | the selected design to scale, with its dimensions (Figure 5) |
| `gallery`, `renders` | every profile at the design fineness with the payload; 3D views |
| `fields`, `grid` | CFD pressure fields around each profile; the CFD grid |

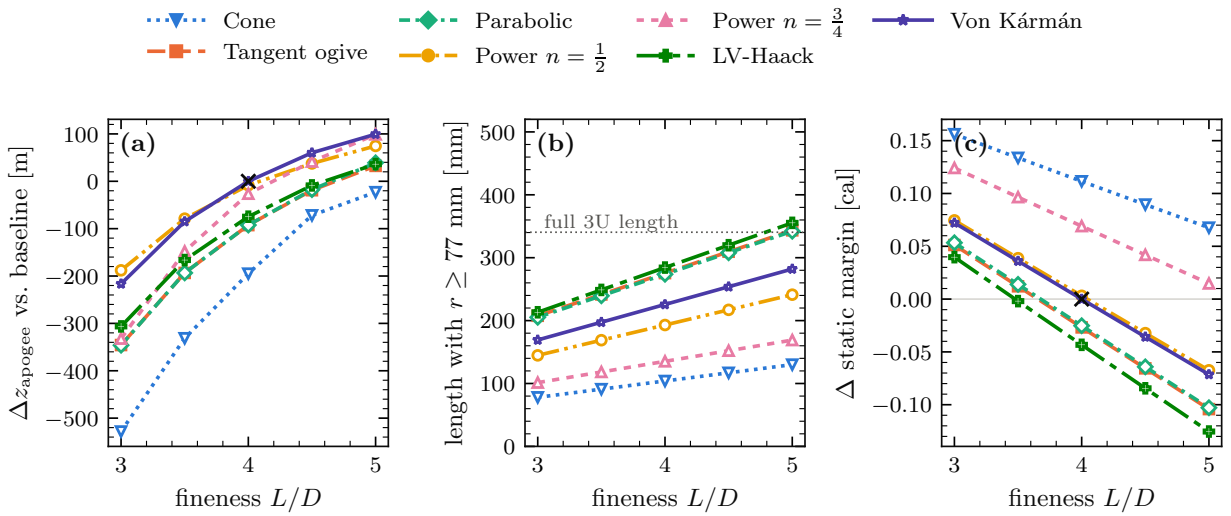



Figure 2: `trade`: (a) apogee change against the baseline (cross), (b) payload length inside the nose, (c) static-margin change. Look for the highest profile in (a) with an acceptable (b).

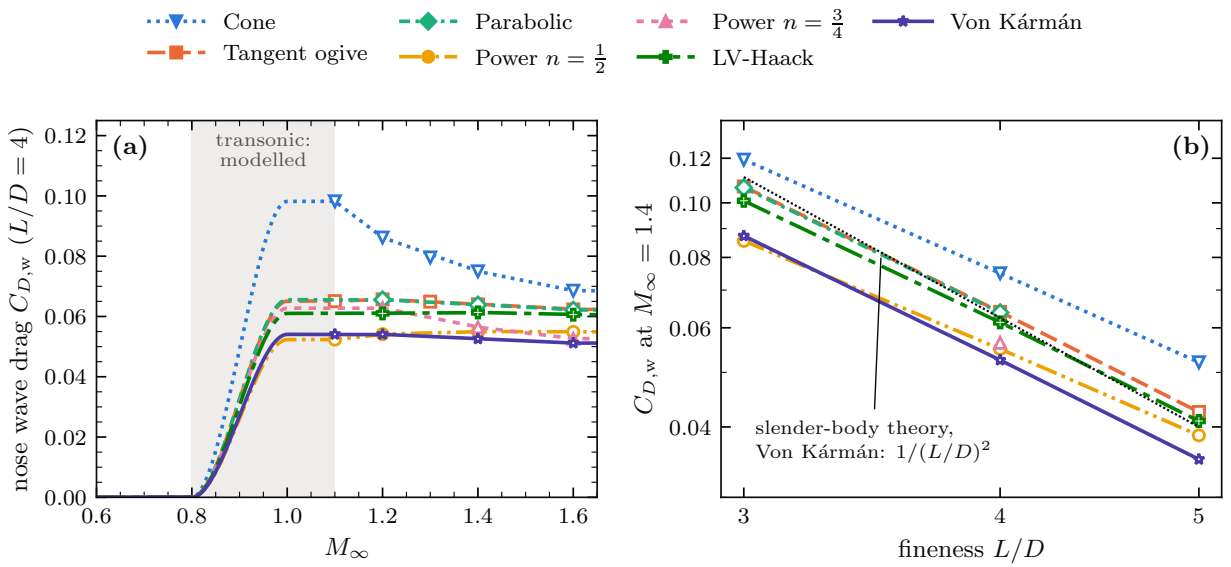



Figure 3: `cdw`: (a) wave drag against Mach at the baseline fineness (markers: CFD; lines: model; grey band: assumed transonic rise); (b) against fineness at $M = 1.4$. Lower is better.

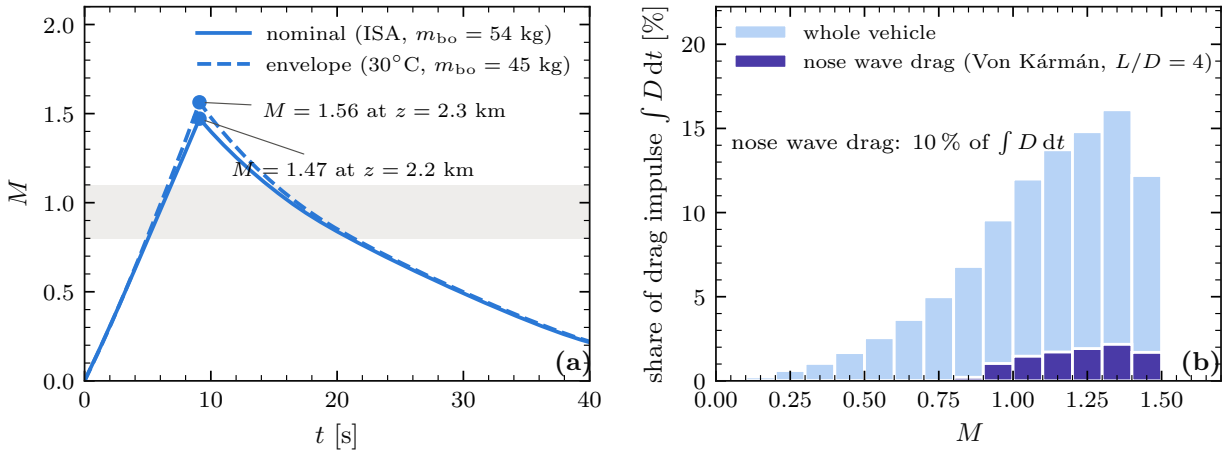



Figure 4: `trajectory`: (a) Mach against time, nominal and hot day; (b) momentum lost to drag per Mach interval, with the nose wave-drag part dark. The nose shape matters only where the dark bars are.

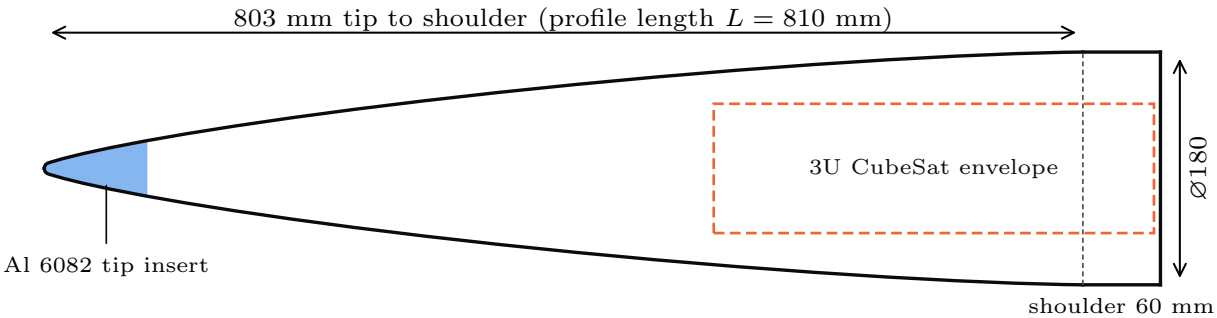



Figure 5: `final`: the selected design to scale, with tip insert, shoulder and payload envelope.

# 7 Extending the Tool

Rules that keep extensions safe: every input is a configuration parameter, every fixed number is in `constants.py`, quantities are SI, every function has a docstring. After any change, run `python tests/verify_reference.py` (Section 7.4).

## 7.1 Adding a profile family

A new member of an existing family needs no code (`nose.custom_profiles`, Section 4.3). A new *family* is one function, which can live in a plugin file (`plugins/secant_ogive.py`):

```
import numpy as np
from nosecone.geometry import register_family

@register_family("secant_ogive", params={"rho_factor": 1.5}, dimensional=True,
                 doc="Secant ogive: arc of radius rho_factor x the tangent-ogive radius")
def secant_ogive(x, L, R, rho_factor):
    rho = rho_factor * (R**2 + L**2) / (2 * R)
    alpha = np.arccos(np.sqrt(L**2 + R**2) / (2 * rho)) - np.arctan(R / L)
    x = np.clip(x, 0.0, L)
    return np.sqrt(np.maximum(rho**2 - (rho*np.cos(alpha) - x)**2, 0.0)) - rho*np.sin(
    alpha)
```

The function takes NumPy arrays and returns $y(0) = 0$, $y(L) = R$, monotonic (with the default `dimensional=False` it takes $x/L$ and returns $y/R$). Load it with `project.plugins`, declare it in `nose.custom_profiles` and set `aerodynamics.compute_missing: true` for its wave drag (`configs/example_plugin.yaml`); everything else follows automatically.

## 7.2 Adding a plot

A plot is a function of a `PlotContext` (`ctx.cfg`, `ctx.study`, `ctx.trade`, `ctx.traj`) registered in `PLOTS`:

```
# nosecone/plotting/figures.py
def fig_mass(ctx: PlotContext) -> None:
    fig, ax = plt.subplots(figsize=(fs.WIDTH, 0.35 * fs.WIDTH))
    for n in ctx.order():
        c, ls, mk = ctx.sty(n)
        d = ctx.trade[ctx.trade["shape"] == n].sort_values("fineness")
        ax.plot(d.fineness, d.mass_g, color=c, ls=ls, marker=mk, label=ctx.label(n))
    ctx.save(fig, "fig_mass")

PLOTS["mass"] = fig_mass          # "mass" is now valid in output.plots
```

## 7.3 Adding a parameter or changing a model

- **A parameter**: one line in the dataclass of its section (`nosecone/config.py`), e.g. `coupler_mass : float = param(0.120, "kg", "Bonded coupler mass", NONNEG)`; it is then validated, listed by `describe-config` and available as `cfg.structure.coupler_mass`.
- **A coefficient** (friction, base drag, heating): `constants.py`.

- **The drag build-up**: `flight.DragModel.components()`; a new component is added to the total, scaled by $k$ and shown in `breakdown`.
- **The transonic wave drag**: `aerodynamics.WaveDrag.__call__`; with RANS data, add the points to the database and lower `aerodynamics.transonic_match`.

### 7.4 Checking a change

`python tests/verify_reference.py` reruns the Weisshorn study and compares every number, table and CAD file with the reference outputs (`--figures`, `--cfd` add the figures and three CFD cases). A change not meant to alter the results must pass unchanged; if it is meant to, update `tests/reference/` only after the new results have been reviewed.

## A Verification Record

The package was refactored from the scripts that produced the Weisshorn design report (`legacy/scripts/`) and checked against them: the sweep table, $k$, the sensitivities, the LaTeX tables and macros, the CFD database and the profile CSV are byte-identical; the whole CFD database recomputes bit for bit (38 geometries, 126 Mach points); 15 of 16 figures are pixel-identical. The few differences are intentional (full-precision $k$ instead of 4 decimals, a closed ogive STL, a corrected table label) and change no rounded number of the report.

## B Troubleshooting

- **"skipped ...: no CFD data for profile ..."**: use a profile of the database, or set `aerodynamics.compute_missing: true`.
- **"calibration failed: the target apogee ... is not reached"**: check masses, thrust and impulse, widen `flight.calibration_bracket`, or set `flight.calibrate: false`.
- **A plot is skipped or "failed"**: `run.log` says why (missing data, or LaTeX: set `output.text_renderer: mathtext`). A failed plot never stops the run.
- **The figures look different from the report**: LaTeX is not installed, so the built-in renderer is used; the content is the same.
- **Results changed after updating Python packages**: differences of order $10^{-12}$ are expected; install the pinned versions of `requirements.txt` for exact reproduction.
- **The CFD solver "diverged"**: it restarts with a lower CFL (`cfd.fallback_cfl`); if it persists, try a finer grid level or a smaller `cfd.cfl`.